

Machine Vision for Bike Ride Mapping

Introduction

In this project, we generated a map of a bike ride given a point-of-view GoPro video. We used OpenCV to track the angle of the rider's handlebars, using this value to approximate the rider's heading over time to generate a 2D map of their ride. We plotted a ground-truth map using the GoPro's GPS data and used this to validate the map we generated through our processing.

Approaches

Glove Contours

One initial method we pursued to calculate the handlebar angle was by detecting the rider's gloves from each frame of the video. To implement this, we created a mask of the frame, isolating the areas of the rider's red gloves and calculating the average X and Y coordinate of each hand. We used the built-in color picker tool to determine the HSV bounds of the red gloves and created our mask to filter out the other colors in the image.



Figure 1: The original image we used to test this approach

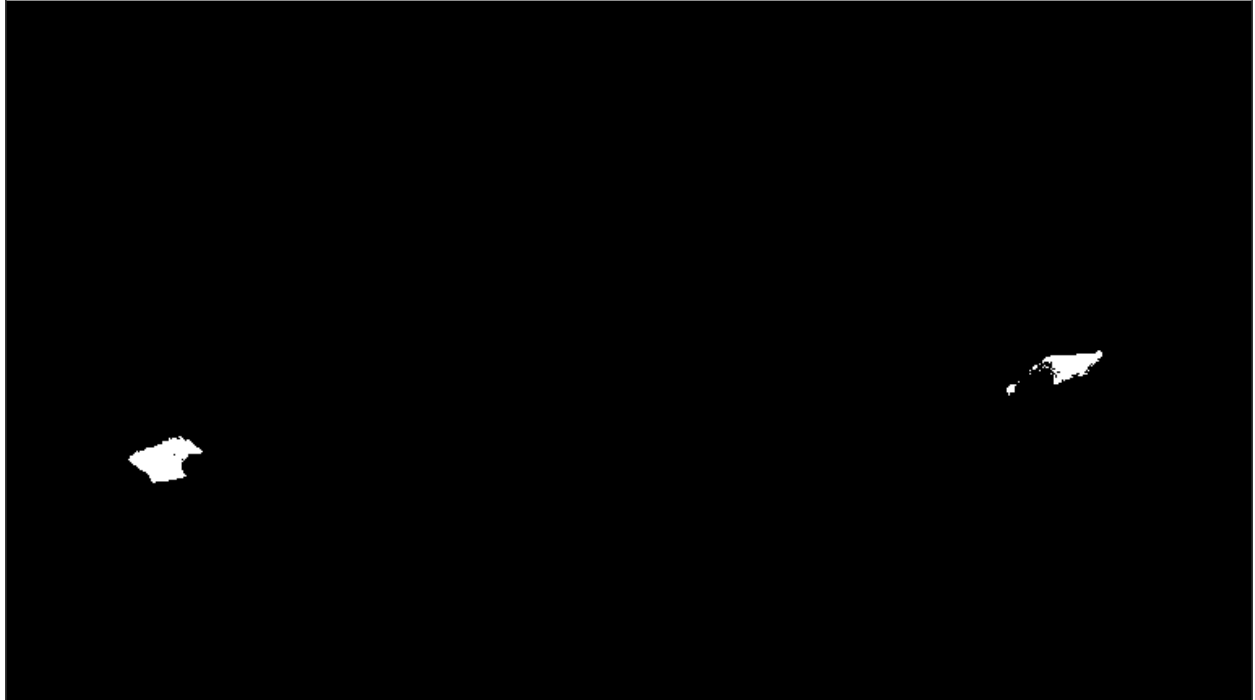


Figure 2: The mask we created by isolating the color of the rider's gloves

Using this binary image of white pixels on a black background, we used the `findContours()` method in openCV to generate a list of numpy arrays of X and Y coordinates for each contour.



Figure 3: Original image with contours overlayed

We then extracted the X and Y coordinates into their own list of arrays of coordinates and split the coordinates between the left glove and right glove using a center X threshold. To split the left and right glove coordinates, we created a list of the minimum values in each array and recorded the index of the first array that contained x coordinates greater than our X coordinate threshold (start of the right glove arrays). Applying this binary mask to our list of all the coordinates, we could separate the contours for the left and right gloves. Since findContours() paired the X and Y coordinates in corresponding arrays, we could apply the same binary mask to the Y coordinates.

After cleaning up the contour arrays and separating them per glove, we simply averaged the X and Y coordinates of all the contours in each glove to calculate the average location of the gloves. We plotted the points and connected them with a line to visualize the angle of the handlebars, shown below.



Figure 4: Original image with approximate angle of handlebars overlaid

Although we didn't end up implementing this approach, we are curious to see how it would perform with our test footage, as it doesn't require a full-on tracker and simply uses a mask with contour detection.

Handlebar Edge Detection

This was the method we ended up implementing, as it was a more robust method of estimating the angle of the handlebars. This method didn't require a specific glove color or other vibrant patch in the image to still track the movement of the ends of the handlebars. This is because we just use one of the included OpenCV trackers, either CSRT or KCF (depending on the video), to track each side of the handlebars and estimate the angle. However, this method is not seamless; we still needed to tweak the location of the initial bounding box estimate before each new video was modeled on to account for differing kinds of POV footage.

GoPro Ground Truth Generation

To compare the map we created from the video data to a known path from the GoPro, we needed a way to extract a map from the latitude and longitude measurements over time. We used an open-source tool to extract all the GPS data from a GoPro MP4 video and record it into a CSV file. Using the Pandas library, we extracted the latitude and longitude values into their own lists. We then used the pyproj library to project the list of latitudes. and longitude values into usable values in meters to plot a map of the ride.

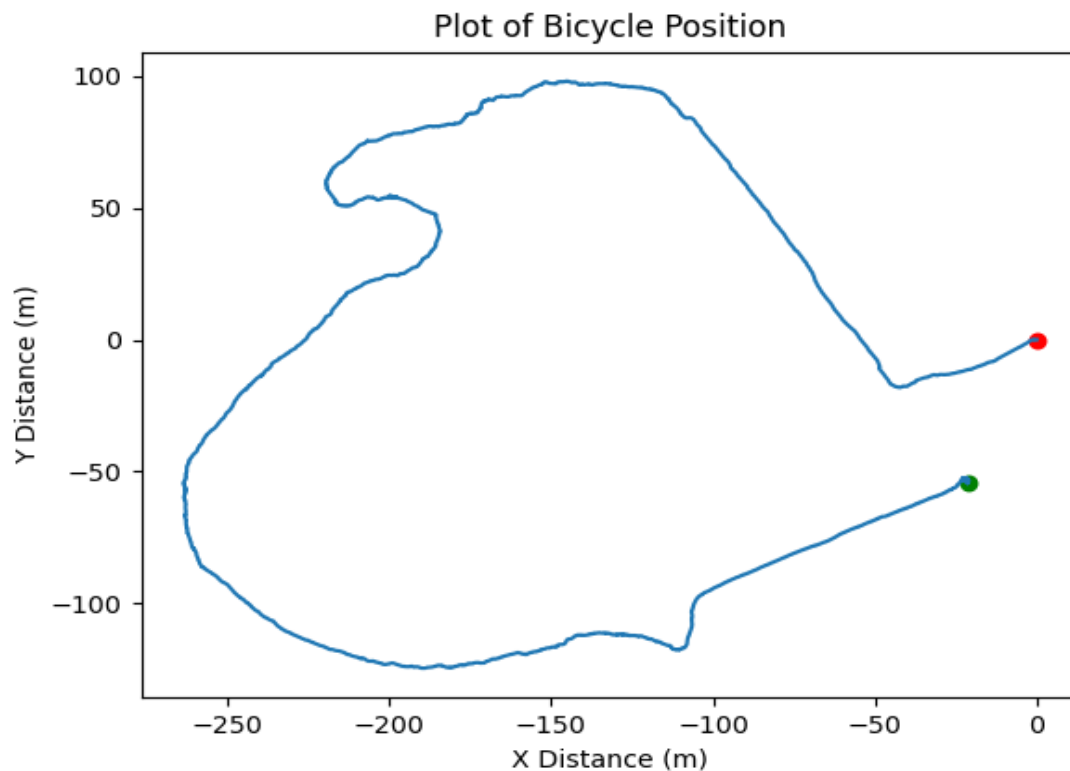


Figure 5: Ground truth map from GoPro GPS data

Implementation

Angle Approximation Using Object Tracking

Our model runs on the assumption that we can get the approximate angle of the handlebars each frame. So, how do we do that? At a high level, we track each side of the handlebars, edge detect each of the bounding boxes created by the trackers, then run line detection after. With our calibrated bounds, only strong lines come through, specifically those made by black handlebars. From those lines, we get an average point for each side and make a line between them. The angle between this line and horizontal is the angle we're looking for. Now, let's dive in a bit deeper.



Figure 6: Photo of the System with Bounding Boxes and Lines Shown

Starting with the trackers (the blue boxes above). We used both Channel and Spatial Reliability Tracking (CSRT) and Kernelized Correlation Filters (KCF) trackers for this project, changing which tracker we used based on the video. By default, we used the CSRT tracker. This is an advanced tracker that has a higher accuracy than most, and does well in challenging environments. This tracker was used, and worked well, for the vast majority of our SamplePOVs. However, there were a couple instances where we decided to use the KCF tracker instead. One issue with the CSRT tracker is that it is much slower than a simpler algorithm, like LCF. For longer SamplePOVs, like the one we used for our GPS comparison, the CSRT takes an order of magnitude longer to run than the LCF tracker. Another problem with the CSRT tracker is that it dynamically adapts and changes the size of the bounding box it is tracking. This is optimal for tracking objects that are rotating and changing a lot in the camera's POV. However, it did cause occasional issues with some of our SamplePOVs. For <10% of our SamplePOVs, one of the CSRT bounding boxes would shrink and shrink until it eventually erred in our system as no lines could be made in it. Again, this was rare, but seemed to happen more with chaotic backgrounds and shaded trails in particular. In either of these cases where CSRT didn't perform, we switched to KCF, which is a constant size bounding box, and runs significantly faster than CSRT.

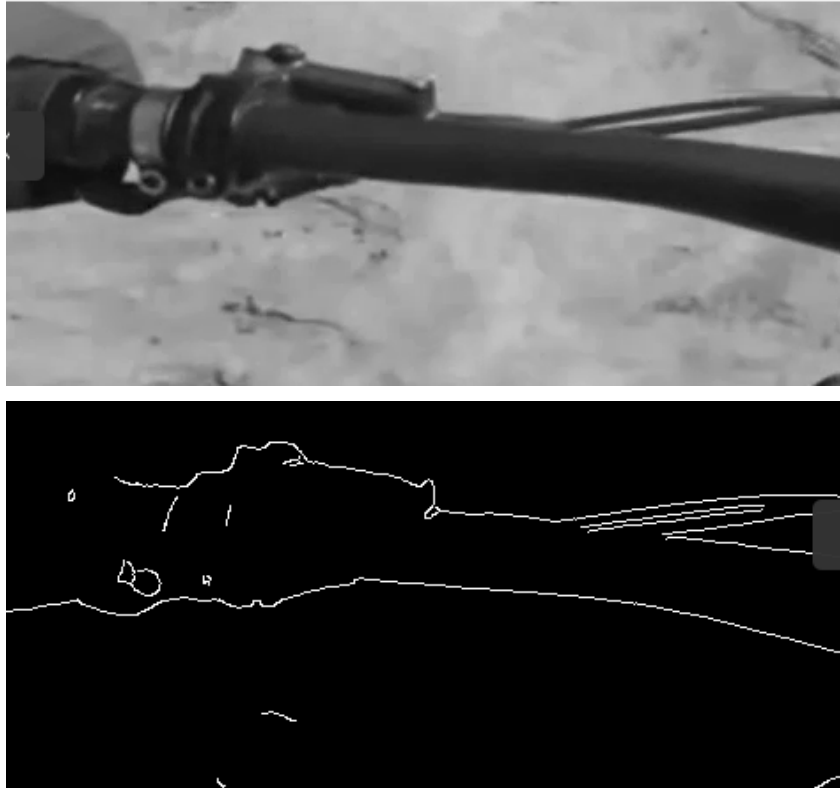


Figure 7: Grayscale Bounding Box (Upper) and Edge Detection (Lower)

Next, we converted each bounding box to greyscale in order to run edge detection. Before running edge detection, we put each grey ROI through a Gaussian Blur. This was used to remove the tiny lines/edges that the edge detection would pick up, which were especially prevalent in tarmac. This blurred data was then fed into a Canny Edge Detector, which was in turn fed into a Hough Line Detector, with quite strict settings. This would usually output 3 to 8 lines, all of which would be focused on the handlebars.



Figure 8: Line Detection with a Relatively Plain Background



Figure 9: Line Detection with a Chaotic Background

For videos where the rider passed from shaded to sunny trail, or chaotic backgrounds, these bounds would need to be decreased, as when the tracker passed such a change, it would tend to pick up no lines and would not work. As such, some videos required lower bounds and had more lines, but still worked reasonably well. In the two figures above, which are taken from the same run with the same settings, we can see how the edge/line detection behaves when comparing the plain and chaotic backgrounds. In the upper one, with plain dirt, the line detection is high perfect, but the lower one, with a tree and some dead wood in the background, behaves worse. It should be noted that it still makes a reasonable approximation though, which is an advantage of how we get the “average” point.

In order to turn the lines into a single point from which to draw our line, we needed a method to do so. In the end, the most effective and accurate method we used was simply taking the average of the y-values for the y value of the point, and setting the X value to a constant relative to the edge of the bounding box. This detail is important, as it's not to say that the X in the world frame doesn't move, it just moves with the bounding box as it tracks the bars. This results in much stable and accurate handlebar approximations than the average point, including averaging the X values.

This process gives us a list of angles for each frame that we can pair with velocity data to construct a map of the trail.

Mapping Approximation

Now we have a list of thetas for each frame. How do we turn that into a map?

To approximate the map/trail, we move in timesteps, placing points along the map. These points are made with small arcs, as this is the way that a bike moves. With the bars turned at a constant angle, the bike will follow a certain, predictable radius of arc. Our timestep will be one frame of the video, which is 30fps, so 33 millisecond steps.

We can calculate the turning radius of the bike with the equation below, where L is the wheelbase (distance between where each of the tires contacts the ground), θ is the angle of the handlebars (0 is straight), and R is the turning radius.

$$R = \frac{L}{\tan(\theta)}$$

We can also calculate the distance we are moving in this timestep, where $V(t)$ is the velocity at the time, and h is our timestep.

$$d = V(t)h$$

From here, we can calculate our change in heading, $\Delta\phi$. It is important to note that $\phi \neq \theta$. These are two different angles.

$$\Delta\phi = \frac{d}{R}$$

Now we know where it's going. We just need to convert it into X and Y in the global frame. Once we know how much the heading changes over the timestep, we can figure out how the bike's position moves on the map. In our approximation, in the bike's local frame, each step simply moves it forward by d . But our map is in a global frame, so that local movement needs to be rotated based on the bike's current heading to find where it ends up in the world. This system means that keeping the handlebars at a constant angle will result in a smooth curve on the global map.

We update the bike's global position and heading at each frame, forming a continuous path that decently approximates the trail the bike actually followed.

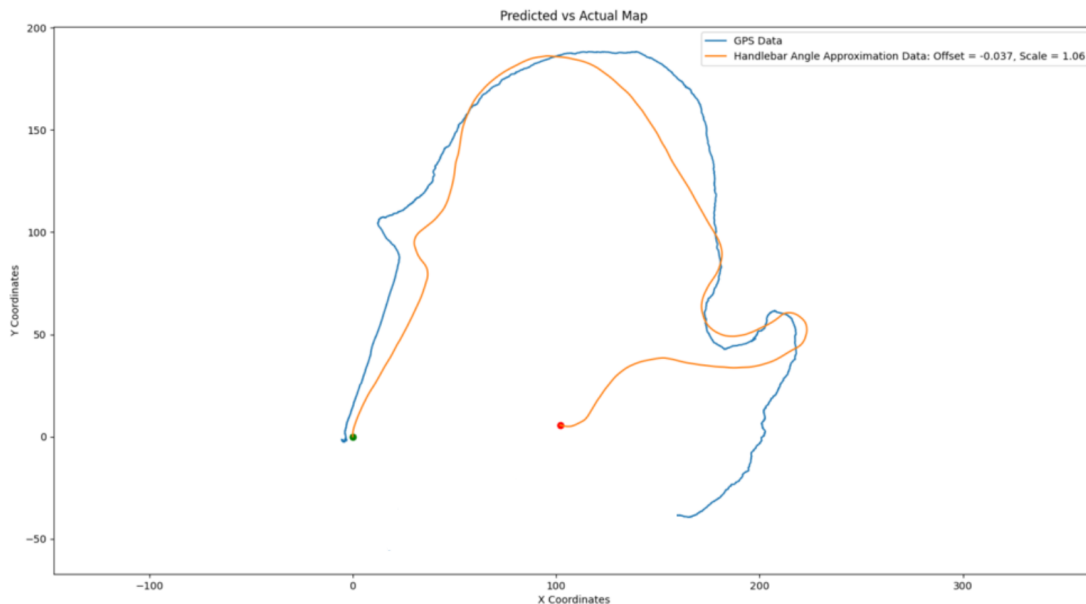


Figure 10: Predicted Angle Approximation Map vs. GPS Data

As you can see in the figure above, our Angle Approximation Model did a decent job of creating a map of the path. It captures features quite well, with each notable turn clearly visible between the two plots above. One issue is location error. Towards the end, the location error is quite large. This is due to the simplicity of our model and the fact that we only have two “knobs” to turn, which we will get into later. Overall though, this was an effective approximation for trail mapping.

Limitations and Challenges

We ran into several issues with the sample videos we ran our model on. One of the struggles we had was finding a POV video we could test on, with slow enough riding to allow our model to keep up while keeping the handlebars in frame for the entire clip. For our final implementation, we recorded our own video with the handlebars in frame around a known path of Olin’s campus with the GoPro’s GPS enabled. Since we were limited to just the raw video data, we couldn’t use any measurements, such as the velocity of the rider, to assist us in creating our map. We could, however, use this data as well as GoPro GPS data to create a validation map to tune our model.

We made a few abstractions to approximate the bike’s heading and create a map of the ride. Our most crucial approximation was how the handlebar angle translated into bike heading using a transfer function. We simplified all the dynamics of the bike (lean, angle of the front wheel, and changing forward velocity of the rider) into a kinematic formula to calculate the static turning radius of the bike using its wheelbase and handlebar angle at each frame. This approach allowed us to approximate the bike’s heading using only data extracted from the video, avoiding the complex dynamic modeling that would require unattainable measurements of the rider or the bike.

One key aspect of turning on a bike is leaning into turns, which we did not account for in our approximation. Taking this roll axis out of our model meant that sharper turns in real life would appear wider in our map approximation because the rider would be leaning and turning the handlebars to a lesser degree. Our final control loop around Olin minimized how much the rider leaned into each turn, staying as upright as we could. Riding upright on the bike and keeping the handlebars in frame for the entire clip was another limitation of our algorithm, as POV mountain bike footage is very dynamic with riders constantly shifting their body position to adjust for different features on the trail. We attempted to run our model on fast-paced downhill footage, but the constant camera movements made tracking very difficult. Since we set the rider’s velocity to a constant value throughout the entire course of the video and constrained ourselves to solely video data, we ignored the effect the rider’s changing velocity would have on the turning radius of the bike.

In addition to these abstractions, our method of fine-tuning our resulting map relied on two parameters: applying an offset value to all the handlebar angles and scaling the steering input angle. The process of tuning these values was very tedious and time-consuming as we needed to adjust the values, run the entire clip through our model, assess the map against the

map we created from the GPS coordinates, and repeat the process. A method to sweep through a wide range of parameter values and assess the fit between our measured versus our predicted map would have been a better solution to these manual tweaks if we were to continue.

Improvements

If we continue with this project and improve its functionality, we would like to either adjust our tracking algorithm, fit a more suitable tracking method, or develop our own algorithm from basic methods to track a wider variety of videos. Creating a general-purpose tracking algorithm that would dynamically adjust to inconsistent video inputs and handle many more riding scenarios, including Z-axis data, is the most robust solution to our overarching goal.

For our current model, we identified several issues that increased computational power and error in our generated map. All the clips we used to test our model had changing backgrounds in each frame, generating noise in our tracking data and therefore adding uncertainty to our line detection algorithm. In the case that there was a consistent background, a distinct object in the background of the frame behind the handlebars, such as a curb, would cause our model to lock onto it rather than the handlebars, shifting our measured handlebar angle for those frames in the clip. To reduce this noise, we believe a method to separate the foreground (handlebars) from the background (dirt, roots, pavement, bricks) would help our model stay consistent on the contours of the handlebars in more cases. To create this mask, we might train a model on POV footage of handlebars to more reliably detect and distinguish the handlebars more generally.

Since we didn't perform any camera calibration, the distortion from the GoPro's wide lens likely skewed our angle data. Using the length and rise of the handlebars, we could improve the accuracy of the handlebar angle by undistorting the footage. This is especially important because we are measuring off the handlebar ends, where the distortion is greatest.

To further improve tracking and potentially generate an estimate of the rider's velocity, we could use a method like visual odometry to track the same keypoints from one frame to the next and calculate the displacement of the rider between consecutive frames. We could implement this as an input in our transfer function to further fine-tune the true heading of the bike over time. To add more depth to our estimate of the rider's heading, we may be able to estimate the lean angle of the bike as well, taking into account the horizon line of the video clip and using the perspective of the camera in relation to this, although this measurement may contain error due to camera mounting angles and the rider's body position not correlating with the roll of the bike. To potentially minimize this, we could rigidly mount the camera to the frame, calibrate its angle relative to the lean angle of the bike, and use it as a reference point for our measurements.

Lessons

We learned many things from this project that we can definitely apply to future projects, in this class, other classes, and in our careers.

- One thing we learned is how effective an extreme abstraction can be. This model is almost as abstracted as it can be. We decided to avoid making a complicated, more realistic model in favor of a simpler one and it paid off quite well. Even though there is location error, especially as time goes on, it captures features, such as turns extremely well. We can apply this kind of approach in the future to get a quick baseline for a model and may even be “good enough” for what we need.
- Initially attempting many different methods of solving the same problem simultaneously is extremely valuable, especially when learning a subject for the first time. The team can explore different paths and come up with solutions that can be applied to the chosen pathway
- Reading the documentation is invaluable. There are so many examples and time-saving functions that can simplify your code and data processing rather than pseudocoding and implementing each step individually.
- When you are learning a new skillset, it can be helpful to create with what you know and what makes sense to you initially before jumping into more complex algorithms and methods to tackle the problem.
- OpenCV is an extremely valuable resource. Having complex computer vision algorithms being completely free and open source is something I never realized was available before. This shows the value in implementing open source libraries as you can often find something online before writing a load of complex code yourself.
- Running frames through a blur can be helpful for edge detection tasks. A simple Gaussian blur completely fixed our tarmac-line detecting issue.
- Getting an MVP done before Demo Day was extremely rewarding and helped us actually reach some of our stretch goals like comparing our approximation to actual GPS data.
- Figure out the math before you figure out the data. We started “at the start” with getting the angle data from the frames. However, when we got to turning this data into a map, we had to go back and change things. A smarter way to do it would be to figure out the data transfer functions first, so you know exactly what you need to get from the data.